



**Department of Electrical,
Computer, & Biomedical Engineering**
Faculty of Engineering
& Architectural Science

Course Title:	Microprocessor Systems
Course Number:	COE538
Semester/Year:	F2025

Instructor:	Dr. Lev Kirischian
--------------------	--------------------

<i>Assignment Number/Lab Number:</i>	Project
<i>Assignment/Lab Title:</i>	Robot Guidance Challenge

<i>Submission Date:</i>	26 November 2025
<i>Due Date:</i>	27 November 2025

Student LAST Name	Student FIRST Name	Student Number	Section	Signature*
Phan	Henry	501232910	07	<i>Henry</i>
Zenko	Jason	501229354	07	JZ
Canning	Harrison	501240661	07	HC

*By signing above you attest that you have contributed to this written lab report and confirm that all work you have contributed to this lab report is your own work. Any suspicion of copying or plagiarism in this work will result in an investigation of Academic Misconduct and may result in a “0” on the work, an “F” in the course, or possibly more severe penalties, as well as a Disciplinary Notice on your academic record under the Student Code of Academic Conduct, which can be found online at: <https://www.torontomu.ca/content/dam/senate/policies/pol60.pdf>

INTRODUCTION

The objective of this project is to design and implement a complete navigation and maze-solving system for the eebot using the HSC12 microcontroller. In this challenge, the robot must travel from the start of the maze to the end autonomously by following a guidance line. Correctly handling each intersection and making branch decisions at each L and T junction. The robot must also demonstrate learning behaviour by exploring dead ends and using the bumper to perform 180 turns, backtracking to choose the alternative path. To accomplish this behaviour, the system integrates multiple hardware and software subsystems, including guider photodetectors for line tracking, bumper switches for obstacle detection, rotation counters for controlled turning, and motor-drive routines for differential steering. The project also requires a structured software architecture involving a navigation manager, guider-processing routines, sensor scanning, motor control, and an organized set of data structures for storing intersection decisions during maze exploration. Overall, the project applies core microprocessor concepts to a real robotic task and provides hands-on experience in system decomposition, state-machine design, algorithmic problem solving, as well as hardware and software integration.

CODE DESCRIPTION

Variables:

```

;*****
; equates section
;*****

; Liquid Crystal Display Equates
;-----
CLEAR_HOME    EQU    $01           ; Clear the display and home the cursor
INTERFACE     EQU    $38           ; 8 bit interface, two line display
CURSOR_OFF    EQU    $0C           ; Display on, cursor off
SHIFT_OFF     EQU    $06           ; Address increments, no character shift
LCD_SEC_LINE  EQU    64           ; Starting addr. of 2nd line of LCD (note decimal value!)

; LCD Addresses
;-----
LCD_CNTR      EQU    PTJ           ; LCD Control Register: E = PJ7, RS = PJ6
LCD_DAT       EQU    PORTB        ; LCD Data Register: D7 = PB7, ... , D0 = PB0
LCD_E         EQU    $80           ; LCD E-signal pin
LCD_RS        EQU    $40           ; LCD RS-signal pin

; Other codes
;-----
NULL          EQU    00           ; The string ?null terminator?
CR            EQU    $0D          ; ?Carriage Return? character
SPACE         EQU    ' '          ; The ?space? character

; Timers
;-----
T_LEFT        EQU    8
T_RIGHT       EQU    8

; States for robot
;-----
START         EQU    0
FWD           EQU    1
ALL_STOP      EQU    2
LEFT_TRN      EQU    3
RIGHT_TRN     EQU    4
REV_TRN       EQU    5
LEFT_ALIGN    EQU    6
RIGHT_ALIGN   EQU    7

; variable/data section
;-----
                ORG    $3800

```

```

; Initial values based on the initial readings & variance
; -----
BASE_LINE      FCB    $96
BASE_BOW       FCB    $CE
BASE_MID        FCB    $CE
BASE_PORT       FCB    $CD
BASE_STBD       FCB    $CD

LINE_VARIANCE   FCB    $18      ; Adding variance based on testing to
BOW_VARIANCE    FCB    $30      ; Establish baseline for sensors
PORT_VARIANCE   FCB    $20
MID_VARIANCE    FCB    $20
STARBOARD_VARIANCE FCB    $15

TOP_LINE        RMB    20      ; Top line of display
                FCB    NULL    ; terminated by null

BOT_LINE        RMB    20      ; Bottom line of display
                FCB    NULL    ; terminated by null

CLEAR_LINE      FCC    '      ' ; Clear the line of display
                FCB    NULL    ; terminated by null

TEMP            RMB    1      ; Temporary location

; Storage Registers (9S12C32 RAM space: $3800 ... $3FFF)
; -----
SENSOR_LINE     FCB    $01      ; Storage for guider sensor readings
SENSOR_BOW      FCB    $23      ; Initialized to test values
SENSOR_PORT     FCB    $45
SENSOR_MID      FCB    $67
SENSOR_STBD     FCB    $89
SENSOR_NUM      RMB    1

; variable section
; *****
TOF_COUNTER     ORG    $3850      ; Where our TOF counter register lives
CRNT_STATE      dc.b  0          ; The timer, incremented at 23Hz
T_TURN         ds.b  1          ; Current state register
TEN_THOUS       ds.b  1          ; time to stop turning
THOUSANDS       ds.b  1          ; 10,000 digit
HUNDREDS        ds.b  1          ; 1,000 digit
TENS            ds.b  1          ; 100 digit
UNITS           ds.b  1          ; 10 digit
NO_BLANK        ds.b  1          ; 1 digit
HEX_TABLE       FCC    '0123456789ABCDEF' ; Used in ?leading zero? blanking by BCD2ASC
BCD_SPARE       RMB    2          ; Table for converting values

; data section
; *****
msg1            dc.b  "Battery volt ",0
msg2            dc.b  "State",0
tab             dc.b  "start",0
                dc.b  "fwd",0
                dc.b  "all_stp",0
                dc.b  "LeftTurn ",0
                dc.b  "RightTurn ",0
                dc.b  "RevTrn ",0
                dc.b  "LeftTimed ",0
                dc.b  "RTimed ",0

```

In this section, these are the variables which will be used across the code. The variables that use *EQU* will define all the constant values, which are used for things such as the LCD control, timers for the ticks of each turn, and states of the eebot. For these variables that use *EQU*, they are fixed and not changed throughout the execution of the program. For the rest of the variables, they are assigned values and

Dispatcher:

```

; subroutine section
; ****
DISPATCHER      JSR    VERIFY_START      ; Start of the Dispatcher
                  RTS

VERIFY_START     CMPA   #START              ; Verify if the robot's state is START
                  BNE   VERIFY_FORWARD     ; If not, move to FORWARD state validation
                  JSR   START_ST           ; Validate START state
                  RTS

VERIFY_FORWARD   CMPA   #FWD                ; Verify if the robot's state is FORWARD
                  BNE   VERIFY_STOP        ; If not, move to ALL_STOP state validation
                  JSR   FWD_ST             ; Validate FORWARD state
                  RTS

VERIFY_REV_TRN   CMPA   #REV_TRN            ; Verify if the robot's state is REV_TURN
                  BNE   VERIFY_LEFT_ALIGN  ; If not, move to LEFT_ALIGN state validation
                  JSR   REV_TRN_ST         ; Validate REV_TURN state
                  RTS

VERIFY_STOP      CMPA   #ALL_STOP           ; Verify if the robot's state is ALL_STOP
                  BNE   VERIFY_LEFT_TRN    ; If not, move to LEFT_TURN state validation
                  JSR   ALL_STOP_ST        ; Validate ALL_STOP state
                  RTS

VERIFY_LEFT_TRN  CMPA   #LEFT_TRN           ; Verify if the robot's state is LEFT_TURN
                  BNE   VERIFY_RIGHT_TRN   ; If not, move to RIGHT_TURN state validation
                  JSR   LEFT               ; Validate LEFT_TURN state
                  RTS

VERIFY_LEFT_ALIGN CMPA   #LEFT_ALIGN        ; Verify if the robot's state is LEFT_ALIGN
                  BNE   VERIFY_RIGHT_ALIGN ; If not, move to RIGHT_ALIGN state validation
                  JSR   LEFT_ALIGN_DONE    ; Validate LEFT_ALIGN state
                  RTS

VERIFY_RIGHT_TRN CMPA   #RIGHT_TRN          ; Verify if the robot's state is RIGHT_TURN
                  BNE   VERIFY_REV_TRN     ; If not, move to REV_TURN state validation
                  JSR   RIGHT              ; Validate RIGHT_TURN state
                  RTS

VERIFY_RIGHT_ALIGN CMPA   #RIGHT_ALIGN      ; Verify if the robot's state is RIGHT_ALIGN
                  JSR   RIGHT_ALIGN_DONE    ; Validate RIGHT_ALIGN state
                  RTS                      ; INVALID state

```

In this section, the dispatcher is the main decision-maker, determining which subroutine to execute. In this code, the dispatcher first goes to *VERIFY_START* to start the dispatcher, then it will use *CMPA* to compare the current state with the next defined state constant. If it is verified that they are equal, it will call the subroutine for that state, but if they are not equal, it will use *BNE* to move to the next state and compare again. This type of comparison will continue until the dispatcher determines which state it is in and which subroutine to execute.

Movement:

```

;Movement
;*****
START_ST      BRCLR   PORTAD0, %00000100,RELEASE
              JSR     INIT_FWD
              MOV     #FWD, CRNT_STATE

RELEASE       RTS

;*****

FWD_ST        BRSET   PORTAD0, $04, NO_FWD_BUMP          ; Checks if bow bumper is hit
              MOV     #REV_TRN, CRNT_STATE              ; if true, enter the
                                                       ; REV_TURN state
              JSR     UPDT_DISPL                        ; Update the display
              JSR     INIT_REV
              LDY     #6000
              JSR     del_50us
              JSR     INIT_RIGHT
              LDY     #6000
              JSR     del_50us
              LBRA    EXIT

NO_FWD_BUMP    BRSET   PORTAD0, $08, NO_FWD_REAR_BUMP      ; Checks if the stern bumper
              MOV     #ALL_STOP, CRNT_STATE             ; if true, enter the
              JSR     INIT_STOP                          ; ALL_STOP state
              LBRA    EXIT

NO_FWD_REAR_BUMP LDAA   SENSOR_BOW
              ADDA    BOW_VARIANCE
              CMPA    BASE_BOW
              BFL     NOT_ALIGNED
              LDAA    SENSOR_MID
              ADDA    MID_VARIANCE
              CMPA    BASE_MID
              BFL     NOT_ALIGNED
              LDAA    SENSOR_LINE
              ADDA    LINE_VARIANCE
              CMPA    BASE_LINE
              BFL     CHECK_RIGHT_ALIGN
              LDAA    SENSOR_LINE
              SUBA    LINE_VARIANCE
              CMPA    BASE_LINE
              BMI     CHECK_LEFT_ALIGN

;*****

NOT_ALIGNED    LDAA    SENSOR_PORT
              ADDA    PORT_VARIANCE
              CMPA    BASE_PORT
              BFL     PARTIAL_LEFT_TRN
              BMI     NO_PORT

NO_PORT        LDAA    SENSOR_BOW
              ADDA    BOW_VARIANCE
              CMPA    BASE_BOW
              BFL     EXIT
              BMI     NO_BOW

NO_BOW         LDAA    SENSOR_STBD
              ADDA    STARBOARD_VARIANCE
              CMPA    BASE_STBD
              BFL     PARTIAL_RIGHT_TRN
              BMI     EXIT

```

```

; *****
PARTIAL_LEFT_TRN  LDY      #6000
                  jsr      del_50us
                  JSR      INIT_LEFT
                  MOVB     #LEFT_TRN, CRNT_STATE
                  LDY      #6000
                  JSR      del_50us
                  BRA      EXIT

CHECK_LEFT_ALIGN  JSR      INIT_LEFT
                  MOVB     #LEFT_ALIGN, CRNT_STATE
                  BRA      EXIT

; *****

PARTIAL_RIGHT_TRN LDY      #6000
                  jsr      del_50us
                  JSR      INIT_RIGHT
                  MOVB     #RIGHT_TRN, CRNT_STATE
                  LDY      #6000
                  JSR      del_50us
                  BRA      EXIT

CHECK_RIGHT_ALIGN JSR      INIT_RIGHT
                  MOVB     #RIGHT_ALIGN, CRNT_STATE
                  BRA      EXIT

EXIT              RTS

; *****

LEFT              LDAA     SENSOR_BOW
                  ADDA     BOW_VARIANCE
                  CMPA     BASE_BOW
                  BPL      LEFT_ALIGN_DONE
                  BMI      EXIT

LEFT_ALIGN_DONE   MOVB     #FWD, CRNT_STATE
                  JSR      INIT_FWD
                  BRA      EXIT

RIGHT             LDAA     SENSOR_BOW
                  ADDA     BOW_VARIANCE
                  CMPA     BASE_BOW
                  BPL      RIGHT_ALIGN_DONE
                  BMI      EXIT

RIGHT_ALIGN_DONE  MOVB     #FWD, CRNT_STATE
                  JSR      INIT_FWD
                  BRA      EXIT

; *****

REV_TRN_ST        LDAA     SENSOR_BOW
                  ADDA     BOW_VARIANCE
                  CMPA     BASE_BOW
                  BMI      EXIT
                  JSR      INIT_LEFT
                  MOVB     #FWD, CRNT_STATE
                  JSR      INIT_FWD
                  BRA      EXIT

ALL_STOP_ST        BRSET   PORTAD0, %00000100, NO_START_BUMP
                  MOVB     #START, CRNT_STATE

NO_START_BUMP     RTS

```

For this section, these are the subroutines that will activate the movements accordingly. It incorporates the sensor readings and bumpers to perform different movements.

For the ***START_ST***, it will activate depending on the input of the bow bumper. If the bow bumper is pressed, it will stay in a “ready” state. When the user releases the bow bumper, it will jump to the ***INIT_FWD*** subroutine, where it will turn on both motors and set its direction to go forward.

For the **FWD_ST**, it will first check the bow bumper to see if it has been hit. If it has been hit, it will trigger *REV_TRN*, update the LCD, perform *INIT_REV*, delay for a fixed time of a 180-degree turn, then exit the state. If the bow bumper has not been hit, it will go to *NO_FWD_BUMP*. It will check if the stern bumper has been hit and if it hasn't, continue to *NO_FWD_REAR_BUMP*. Here, it will check the threshold of the front sensor by adding the *BOW_VARIANCE* to *SENSOR_BOW* and comparing it with *BASE_BOW*. If it is greater than or equal to the value of *BASE_BOW*, it will align the eebot by using the *NOT_ALIGNED* subroutine, where the program will choose *PARTIAL_LEFT_TRN* or *PARTIAL_RIGHT_TRN*, depending on which side is brighter when the sensors detect the lines. After, it will check that turn with *CHECK_LEFT_ALIGN* and *CHECK_RIGHT_ALIGN* to make alignments. If it is less than the value of *BASE_BOW*, it will check the other sensors and continue the same process of aligning it so that the eebot is going in the right path. As the eebot goes through the maze, it will see which side has more of the "line" of the maze through its sensors and determine whether to turn left or right.

For **LEFT** and **RIGHT**, it will activate these states when the dispatcher reaches the left or right turn states. Here, it will check the sensors and compare with the *BASE_BOW* threshold. If the reading is higher than or equal to the threshold, it will branch to *LEFT_ALIGN_DONE* for left and *RIGHT_ALIGN_DONE* for right. If it is lower, this means that it's not aligned, and it will not leave the state until it is aligned.

For **REV_TRN_ST**, it will adjust the 180-degree turn to make sure it is properly aligned. If the value compared to the *BASE_BOW* is higher than or equal to the threshold, it will initiate that left turn to align itself, then change the state back to FWD, and move forward again. If the value is less than the threshold, it will continue to turn until the threshold is reached.

Initialization:

```

; Initialization Subroutines
; ****
INIT_RIGHT      BSET    PORTA,%00000010
                BCLR    PORTA,%00000001
                LDAA    TOF_COUNTER          ; Mark the fwd_turn time Tfdturn
                ADDA    #T_RIGHT
                STAA    T_TURN
                RTS

INIT_LEFT       BSET    PORTA,%00000001
                BCLR    PORTA,%00000010
                LDAA    TOF_COUNTER          ; Mark TOF time
                ADDA    #T_LEFT              ; Add left turn
                STAA    T_TURN
                RTS

INIT_FWD        BCLR    PORTA,%00000011      ; Set FWD dir. for both motors
                BSET    PTT,%00110000        ; Turn on the drive motors
                RTS

INIT_REV        BSET    PORTA,%00000011      ; Set REV direction for both motors
                BSET    PTT,%00110000        ; Turn on the drive motors
                RTS

INIT_STOP       BCLR    PTT,%00110000        ; Turn off the drive motors
                RTS

; ****
; Initialize Sensors
;
INIT            BCLR    DDRAD,$FF ; Make PORTAD an input (DDRAD @ $0272)
                BSET    DDRA,$FF  ; Make PORTA an output (DDRA @ $0002)
                BSET    DDRB,$FF  ; Make PORTB an output (DDRB @ $0003)
                BSET    DDRJ,$C0  ; Make pins 7,6 of PTJ outputs (DDRJ @ $026A)
                RTS

; ****
; Initialize ADC
;
openADC         MOVW    #$80,ATDCTL2 ; Turn on ADC (ATDCTL2 @ $0082)
                LDY     #1           ; Wait for 50 us for ADC to be ready
                JSR     del_50us     ; - " -
                MOVW    #$20,ATDCTL3 ; 4 conversions on channel AN1 (ATDCTL3 @ $0083)
                MOVW    #$97,ATDCTL4 ; 8-bit resolution, prescaler=48 (ATDCTL4 @ $0084)
                RTS

```

For this section, this is where the initialization of the movements will be created. For INIT_RIGHT, it will set the motors in a position to turn right, then it will load the TOF_COUNTER. After this, it will add the value of T_RIGHT to the counter and load it into T_TURN. For INIT_LEFT, it will be the same process as the right turn, but for turning left. For INIT_FWD, it will clear the bits of PORTA to set the direction to forward and set the bits of PTT to turn the motors on. For INIT_REV, it will do the same as INIT_FWD, but instead of clearing the bits, it will set the direction bits of PORTA so that the motors go in the reverse direction. For INIT_STOP, it will just clear the bits of PTT so that the motors turn off.

Utility Subroutines:

```

initLCD:      BSET    DDRB,%11111111 ; configure pins PS7,PS6,PS5,PS4 for output
              BSET    DDRJ,%11000000 ; configure pins PE7,PE4 for output
              LDY      #2000
              JSR      del_50us
              LDAA      #$28
              JSR      cmd2LCD
              LDAA      #$0C
              JSR      cmd2LCD
              LDAA      #$06
              JSR      cmd2LCD
              RTS

;*****
clrLCD:      LDAA      #$01
              JSR      cmd2LCD
              LDY      #40
              JSR      del_50us
              RTS

;*****
del_50us     PSHX
              LDX      #300           ; (2 E-clk) Protect the X register
eloop        ; (2 E-clk) Initialize the inner loop counter
iloop        NOP                    ; (1 E-clk) No operation
              DBNE     X,iloop        ; (3 E-clk) If the inner cntr not 0, loop again
              DBNE     Y,eloop        ; (3 E-clk) If the outer cntr not 0, loop again
              PULX
              RTS                    ; (3 E-clk) Restore the X register
              ; (5 E-clk) Else return

;*****
cmd2LCD:     BCLR     LCD_CNTR, LCD_RS ; select the LCD instruction
              JSR      dataMov        ; send data to IR
              RTS

;*****
putsLCD:     LDAA      1,X+           ; get one character from string
              BEQ      donePS         ; get NULL character
              JSR      putcLCD
              BRA      putsLCD

donePS       RTS

;*****
putcLCD:     BSET     LCD_CNTR, LCD_RS ; select the LCD data register (DR)c
              JSR      dataMov        ; send data to DR
              RTS

;*****
dataMov:     BSET     LCD_CNTR, LCD_E ; pull LCD E-signal high
              STAA     LCD_DAT        ; send the upper 4 bits of data to LCD
              BCLR     LCD_CNTR, LCD_E ; pull the LCD E-signal low to complete write oper.
              ISLA
              ISLA                    ; match the lower 4 bits with LCD data pins
              ISLA                    ; ""
              ISLA                    ; ""
              ISLA                    ; ""
              BSET     LCD_CNTR, LCD_E ; pull LCD E-signal high
              STAA     LCD_DAT        ; send the lower 4 bits of data to LCD
              BCLR     LCD_CNTR, LCD_E ; pull the LCD E-signal low to complete write oper.
              LDY      #1             ; adding this delay allows
              JSR      del_50us       ; completion of most instructions
              RTS

;*****
initAD       MOVW     #$C0,ATDCTL2    ;power up AD, select fast flag clear
              JSR      del_50us       ;wait for 50 us
              MOVW     #$00,ATDCTL3    ;8 conversions in a sequence
              MOVW     #$85,ATDCTL4    ;res=8, conv-clks=2, prescal=12
              BSET     ATDDIEN,$0C     ;configure pins AN03,AN02 as digital inputs
              RTS

```

In this section, the basics of the LCD setup are initialized, and the delay is made as well. These subroutines are important as they are needed for the LCD control to display the outputs.

TOF:

```

; *****
ENABLE_TOF      LDAA    #%10000000
                  STAA    TSCR1        ; Enable TCNT
                  STAA    TFLG2        ; Clear TOF
                  LDAA    #%10000100    ; Enable TOI and select prescale factor equal to 16
                  STAA    TSCR2
                  RTS

TOF_ISR          INC     TOF_COUNTER
                  LDAA    #%10000000    ; Clear
                  STAA    TFLG2        ; TOF
                  RTI

```

In this section, the *ENABLE_TOF* subroutine will enable the TCNT, then use *TFLG2* to clear the TOF flag. Then, it will enable the TOI and set the prescale factor to 16, so that the tick timing is around 23 Hz. For the *TOF_ISR* subroutine, it will increment the *TOF_COUNTER*, then it will clear the TOF flag.

CHALLENGES

A significant challenge we encountered during the project was the eebot's inability to reliably detect and follow the guidance line. Instead of maintaining its path, the robot frequently drifted away from the track and began free-roaming across the board. This issue made it impossible to test other aspects of the objective, such as intersection handling and turning after hitting a wall. As a result, much of our time was spent diagnosing why the robot failed to maintain lock on the line and determining whether the root cause was hardware-related, software- related or a combination of both.

Another major challenge we encountered was the eebot occasionally overturning during movement or after doing a full turn. This introduced significant instability and positional offset for the rest of the maze. This not only disrupted its ability to follow the intended path but also affected the sensor alignment, causing inaccuracies in line detection and adjustments made later on in that maze run.

We also faced a challenge where the eebot consistently failed to detect a particular section of the maze, causing it to miss a required left turn. This issue created a cascading failure: because the robot did

not register the left turn, it never executed the corresponding right turn later in the maze, even in scenarios where turning right was crucial to completing the maze. This behaviour significantly impacted the reliability of our maze-solving algorithm, as the eebot couldn't pass this barrier right at the end of the maze, causing the eebot to never complete the maze.

SOLUTIONS

To resolve the issue of unreliable line tracking, we determined that the root cause was the dispatcher not reading the sensor states correctly. The dispatcher is responsible for interpreting the eebot's line-sensing inputs and assigning the appropriate behavioural state. Any misreading directly affected the robot's ability to follow the track. After identifying this mismatch, we reviewed and corrected the dispatcher logic to ensure that each sensor state corresponded accurately to the expected navigation behaviour. Once the dispatcher was fixed, the eebot began sensing the line consistently.

To address the instability and overturning issue, we discovered that the time delays programmed for the eebot's turning movements were too long. These extended time delays caused the robot to over-rotate turns and over-correct line following adjustments. After identifying the connection between the delay time and the robot's instability, we recalibrated and reduced the turn timing values. This adjustment allowed the eebot to perform smoother, more controlled turns without excessive rotation.

To resolve the issue of the eebot missing a crucial left turn in the maze, we identified two related problems in the detection and decision-making process. First, the robot was not sensing the path quickly enough due to overly long time delays in the detection loop. These delays caused the eebot to move past the left turn section before the sensors had the chance to read it. To fix this, we reduced the detection delay, allowing the robot to sample the sensor inputs more frequently and detect the left turn in real time.

Second, we discovered that when the left turn condition failed, the program never evaluated the corresponding right turn case because that branch was missing in the state logic. That meant that when the eebot missed a left turn, it wouldn't make the right turn later on to fix the mistake and would only make turns where a left was available. After correcting the right turn decision-making, the robot was able to make accurate left and right turns as needed. Together, these two solutions allowed the robot to handle the section of the maze we had trouble with.

CONCLUSION

Overall, this lab project combined the major skills developed in COE538 into a complete real-time robotics application. By integrating sensor input, motor control, state machine logic, and supporting subroutines, we built a functional navigation system that allowed the eebot to move through the maze, detect intersections, make decisions, and correct its path. Although the system required extensive calibration, especially with sensor thresholds, turn timing, and alignment, the eebot ultimately followed the intended control flow reliably once tuned. The challenges we faced, such as inconsistent line tracking and overturning during turns, underscored the difficulty of coordinating hardware with software. They also highlighted the importance of iterative debugging, careful calibration, and systematic testing. Refining dispatcher logic, adjusting delays, and improving sensor polling reinforced key course principles such as incremental development and modular, well-structured state machine design. Despite these obstacles, the project successfully demonstrated how earlier concepts, including motor control, ADC setup, timing routines, and state-based decision making, can be combined into a working embedded system. It provided valuable hands-on experience with microcontroller programming, real-time control, and full system integration.

APPENDIX

```
;102940 bot
```

```
XDEF Entry, _Startup ;
    ABSENTRY Entry ;
    INCLUDE "derivative.inc"
```

```
.*****
;
; equates section
.*****
```

```
; Liquid Crystal Display Equates
```

```
;-----
CLEAR_HOME EQU $01          ; Clear the display and home the cursor
INTERFACE EQU $38           ; 8 bit interface, two line display
CURSOR_OFF EQU $0C          ; Display on, cursor off
SHIFT_OFF EQU $06           ; Address increments, no character shift
LCD_SEC_LINE EQU 64         ; Starting addr. of 2nd line of LCD (note decimal value!)
```

```
; LCD Addresses
```

```
;-----
LCD_CNTR EQU PTJ            ; LCD Control Register: E = PJ7, RS = PJ6
LCD_DAT EQU PORTB           ; LCD Data Register: D7 = PB7, ... , D0 = PB0
LCD_E EQU $80               ; LCD E-signal pin
LCD_RS EQU $40              ; LCD RS-signal pin
```

```
; Other codes
```

```
;-----
NULL EQU 00                ; The string ?null terminator?
CR EQU $0D                 ; ?Carriage Return? character
SPACE EQU ' '              ; The ?space? character
```

```
; Timers
```

```
;-----
T_LEFT EQU 8
T_RIGHT EQU 8
```

```
; States for robot
```

```
;-----
START EQU 0
FWD EQU 1
ALL_STOP EQU 2
LEFT_TRN EQU 3
```

```

RIGHT_TRN EQU 4
REV_TRN EQU 5
LEFT_ALIGN EQU 6
RIGHT_ALIGN EQU 7

```

```

; variable/data section

```

```

; -----

```

```

    ORG $3800

```

```

; Initial values based on the initial readings & variance

```

```

; -----

```

```

BASE_LINE FCB $96
BASE_BOW FCB $CE
BASE_MID FCB $CE
BASE_PORT FCB $CD
BASE_STBD FCB $CD

```

```

LINE_VARIANCE FCB $18 ; Adding variance based on testing to
BOW_VARIANCE FCB $30 ; Establish baseline for sensors
PORT_VARIANCE FCB $20
MID_VARIANCE FCB $20
STARBOARD_VARIANCE FCB $15

```

```

TOP_LINE RMB 20 ; Top line of display
          FCB NULL ; terminated by null

```

```

BOT_LINE RMB 20 ; Bottom line of display
          FCB NULL ; terminated by null

```

```

CLEAR_LINE FCC ' ' ; Clear the line of display
          FCB NULL ; terminated by null

```

```

TEMP RMB 1 ; Temporary location

```

```

; Storage Registers (9S12C32 RAM space: $3800 ... $3FFF)

```

```

; -----

```

```

SENSOR_LINE FCB $01 ; Storage for guider sensor readings
SENSOR_BOW FCB $23 ; Initialized to test values
SENSOR_PORT FCB $45
SENSOR_MID FCB $67
SENSOR_STBD FCB $89
SENSOR_NUM RMB 1

```

```

; variable section

```

```

.*****
,

```



```

        ORG $3850                ; Where our TOF counter register lives
TOF_COUNTER dc.b 0                ; The timer, incremented at 23Hz
CRNT_STATE  dc.b 2                ; Current state register
T_TURN      ds.b 1                ; time to stop turning
TEN_THOUS    ds.b 1                ; 10,000 digit
THOUSANDS    ds.b 1                ; 1,000 digit
HUNDREDS     ds.b 1                ; 100 digit
TENS         ds.b 1                ; 10 digit
UNITS        ds.b 1                ; 1 digit
NO_BLANK     ds.b 1                ; Used in ?leading zero? blanking by BCD2ASC
HEX_TABLE    FCC '0123456789ABCDEF' ; Table for converting values
BCD_SPARE    RMB 2

```

```

; code section

```

```

.*****
,

```

```

        ORG $4000
Entry:
_Startup:

```

```

        LDS # $4000                ; Initialize the stack pointer
        CLI                        ; Enable interrupts
        JSR INIT                    ; Initialize ports
        JSR openADC                 ; Initialize the ATD
        JSR initLCD                 ; Initialize the LCD
        JSR CLR_LCD_BUF             ; Write ?space? characters to the LCD buffer
        BSET DDRA,%00000011         ; STAR_DIR, PORT_DIR
        BSET DDRT,%00110000         ; STAR_SPEED, PORT_SPEED
        JSR initAD                  ; Initialize ATD converter
        JSR initLCD                 ; Initialize the LCD
        JSR clrLCD                  ; Clear LCD & home cursor
        LDX #msg1                   ; Display msg1
        JSR putsLCD                 ;      "
        LDAA #$C0                   ; Move LCD cursor to the 2nd row
        JSR cmd2LCD                 ;
        LDX #msg2                   ; Display msg2
        JSR putsLCD                 ;      "
        JSR ENABLE_TOF              ; Jump to TOF initialization

```

```

MAIN

```

```

        JSR G_LEDS_ON               ; Enable the guider LEDs
        JSR READ_SENSORS            ; Read the 5 guider sensors
        JSR G_LEDS_OFF              ; Disable the guider LEDs
        JSR UPDT_DISPL
        LDAA CRNT_STATE

```

```

JSR  DISPATCHER
BRA  MAIN

```

```

; data section

```

```

*****
,
msg1      dc.b "Battery volt ",0
msg2      dc.b "State",0
tab       dc.b "start ",0
          dc.b "fwd  ",0
          dc.b "all_stp",0
          dc.b "LeftTurn ",0
          dc.b "RightTurn ",0
          dc.b "RevTrn ",0
          dc.b "LeftTimed ",0
          dc.b "RTimed ",0

```

```

; subroutine section

```

```

*****
,
DISPATCHER  JSR  VERIFY_START          ; Start of the Dispatcher
              RTS

VERIFY_START  CMPA  #START              ; Verify if the robot's state is START
              BNE  VERIFY_FORWARD      ; If not, move to FORWARD state validation
              JSR  START_ST            ; Validate START state
              RTS

VERIFY_FORWARD  CMPA  #FWD              ; Verify if the robot's state is
FORWARD
              BNE  VERIFY_STOP          ; If not, move to ALL_STOP state validation
              JSR  FWD_ST              ; Validate FORWARD state
              RTS

VERIFY_REV_TRN  CMPA  #REV_TRN          ; Verify if the robot's state is
REV_TURN
              BNE  VERIFY_LEFT_ALIGN   ; If not, move to LEFT_ALIGN state
validation
              JSR  REV_TRN_ST          ; Validate REV_TURN state
              RTS

VERIFY_STOP    CMPA  #ALL_STOP          ; Verify if the robot's state is
ALL_STOP
              BNE  VERIFY_LEFT_TRN     ; If not, move to LEFT_TURN state
validation
              JSR  ALL_STOP_ST         ; Validate ALL_STOP state

```

RTS

VERIFY_LEFT_TRN CMPA #LEFT_TRN ; Verify if the robot's state is
LEFT_TURN

BNE VERIFY_RIGHT_TRN ; If not, move to RIGHT_TURN state
validation

JSR LEFT ; Validate LEFT_TURN state
RTS

VERIFY_LEFT_ALIGN CMPA #LEFT_ALIGN ; Verify if the robot's state is
LEFT_ALIGN

BNE VERIFY_RIGHT_ALIGN ; If not, move to RIGHT_ALIGN state
validation

JSR LEFT_ALIGN_DONE ; Validate LEFT_ALIGN state
RTS

VERIFY_RIGHT_TRN CMPA #RIGHT_TRN ; Verify if the robot's state is
RIGHT_TURN

BNE VERIFY_REV_TRN ; If not, move to REV_TURN state validation
JSR RIGHT ; Validate RIGHT_TURN state

VERIFY_RIGHT_ALIGN CMPA #RIGHT_ALIGN ; Verify if the robot's state is
RIGHT_ALIGN

JSR RIGHT_ALIGN_DONE ; Validate RIGHT_ALIGN state
RTS ; INVALID state

;Movement

.*****
,

START_ST BRCLR PORTAD0, %00000100, RELEASE
JSR INIT_FWD
MOVB #FWD, CRNT_STATE

RELEASE RTS

.*****
,

FWD_ST BRSET PORTAD0, \$04, NO_FWD_BUMP ; Checks if bow bumper is
hit

MOVB #REV_TRN, CRNT_STATE ; if true, enter the
; REV_TURN state

JSR UPDT_DISPL ; Update the display
JSR INIT_REV
LDY #6000

```

JSR  del_50us
JSR  INIT_RIGHT
LDY  #6000
JSR  del_50us
LBRA EXIT

```

```

NO_FWD_BUMP    BRSET  PORTAD0, $08, NO_FWD_REAR_BUMP    ; Checks if the
stern bumper

```

```

    MOVB  #ALL_STOP, CRNT_STATE    ; if true, enter the
    JSR  INIT_STOP                ; ALL_STOP state
    LBRA EXIT

```

```

NO_FWD_REAR_BUMP LDAA  SENSOR_BOW

```

```

    ADDA  BOW_VARIANCE
    CMPA  BASE_BOW
    BPL  NOT_ALIGNED
    LDAA  SENSOR_MID
    ADDA  MID_VARIANCE
    CMPA  BASE_MID
    BPL  NOT_ALIGNED
    LDAA  SENSOR_LINE
    ADDA  LINE_VARIANCE
    CMPA  BASE_LINE
    BPL  CHECK_RIGHT_ALIGN
    LDAA  SENSOR_LINE
    SUBA  LINE_VARIANCE
    CMPA  BASE_LINE
    BMI  CHECK_LEFT_ALIGN

```

```

,*****

```

```

NOT_ALIGNED    LDAA  SENSOR_PORT
    ADDA  PORT_VARIANCE
    CMPA  BASE_PORT
    BPL  PARTIAL_LEFT_TRN
    BMI  NO_PORT

```

```

NO_PORT        LDAA  SENSOR_BOW
    ADDA  BOW_VARIANCE
    CMPA  BASE_BOW
    BPL  EXIT
    BMI  NO_BOW

```

```

NO_BOW         LDAA  SENSOR_STBD

```

```

    ADDA  STARBOARD_VARIANCE
    CMPA  BASE_STBD
    BPL   PARTIAL_RIGHT_TRN
    BMI   EXIT

```

```

,*****
,

```

```

PARTIAL_LEFT_TRN LDY  #6000
    jsr  del_50us
    JSR  INIT_LEFT
    MOVB #LEFT_TRN, CRNT_STATE
    LDY  #6000
    JSR  del_50us
    BRA  EXIT

```

```

CHECK_LEFT_ALIGN JSR  INIT_LEFT
    MOVB #LEFT_ALIGN, CRNT_STATE
    BRA  EXIT

```

```

,*****
,

```

```

PARTIAL_RIGHT_TRN LDY  #6000
    jsr  del_50us
    JSR  INIT_RIGHT
    MOVB #RIGHT_TRN, CRNT_STATE
    LDY  #6000
    JSR  del_50us
    BRA  EXIT

```

```

CHECK_RIGHT_ALIGN JSR  INIT_RIGHT
    MOVB #RIGHT_ALIGN, CRNT_STATE
    BRA  EXIT

```

```

EXIT      RTS

```

```

,*****
,

```

```

LEFT      LDAA  SENSOR_BOW
    ADDA  BOW_VARIANCE
    CMPA  BASE_BOW
    BPL   LEFT_ALIGN_DONE
    BMI   EXIT

```

```

LEFT_ALIGN_DONE MOVB  #FWD, CRNT_STATE

```

```

        JSR  INIT_FWD
        BRA  EXIT

RIGHT      LDAA  SENSOR_BOW
          ADDA  BOW_VARIANCE
          CMPA  BASE_BOW
          BPL  RIGHT_ALIGN_DONE
          BMI  EXIT

RIGHT_ALIGN_DONE  MOVB  #FWD, CRNT_STATE
          JSR  INIT_FWD
          BRA  EXIT

.*****
,

REV_TRN_ST  LDAA  SENSOR_BOW
          ADDA  BOW_VARIANCE
          CMPA  BASE_BOW
          BMI  EXIT
          JSR  INIT_LEFT
          MOVB  #FWD, CRNT_STATE
          JSR  INIT_FWD
          BRA  EXIT

ALL_STOP_ST  BRSET  PORTAD0, %00000100, NO_START_BUMP
          MOVB  #START, CRNT_STATE

NO_START_BUMP  RTS

; Initialization Subroutines
.*****
,
INIT_RIGHT    BSET  PORTA,%00000010
          BCLR  PORTA,%00000001
          LDAA  TOF_COUNTER          ; Mark the fwd_turn time Tfwddturn
          ADDA  #T_RIGHT
          STAA  T_TURN
          RTS

INIT_LEFT     BSET  PORTA,%00000001
          BCLR  PORTA,%00000010
          LDAA  TOF_COUNTER          ; Mark TOF time
          ADDA  #T_LEFT              ; Add left turn
          STAA  T_TURN
          RTS

```

```
INIT_FWD      BCLR  PORTA,%00000011      ; Set FWD dir. for both motors
              BSET  PTT,%00110000        ; Turn on the drive motors
              RTS
```

```
INIT_REV      BSET  PORTA,%00000011      ; Set REV direction for both motors
              BSET  PTT,%00110000        ; Turn on the drive motors
              RTS
```

```
INIT_STOP     BCLR  PTT,%00110000        ; Turn off the drive motors
              RTS
```

```
*****
;
```

```
; Initialize Sensors
```

```
INIT          BCLR  DDRAD,$FF ; Make PORTAD an input (DDRAD @ $0272)
              BSET  DDRA,$FF ; Make PORTA an output (DDRA @ $0002)
              BSET  DDRB,$FF ; Make PORTB an output (DDRB @ $0003)
              BSET  DDRJ,$C0 ; Make pins 7,6 of PTJ outputs (DDRJ @ $026A)
              RTS
```

```
*****
;
```

```
; Initialize ADC
```

```
openADC       MOVB  #$80,ATDCTL2 ; Turn on ADC (ATDCTL2 @ $0082)
              LDY   #1          ; Wait for 50 us for ADC to be ready
              JSR   del_50us    ; - " -
              MOVB  #$20,ATDCTL3 ; 4 conversions on channel AN1 (ATDCTL3 @ $0083)
              MOVB  #$97,ATDCTL4 ; 8-bit resolution, prescaler=48 (ATDCTL4 @ $0084)
              RTS
```

```
;-----
```

```
; Clear LCD Buffer
```

```
; This routine writes ?space? characters (ascii 20) into the LCD display
```

```
; buffer in order to prepare it for the building of a new display buffer.
```

```
; This needs only to be done once at the start of the program. Thereafter the
```

```
; display routine should maintain the buffer properly.
```

```
CLR_LCD_BUF   LDX   #CLEAR_LINE
              LDY   #TOP_LINE
              JSR   STRCPY
```

```
CLB_SECOND    LDX   #CLEAR_LINE
              LDY   #BOT_LINE
              JSR   STRCPY
```

CLB_EXIT RTS

```

;
; String Copy
; Copies a null-terminated string (including the null) from one location to
; another
; Passed: X contains starting address of null-terminated string
; Y contains first address of destination
STRCPY      PSHX      ; Protect the registers used
            PSYH
            PSHA

```

```
STRCPY_LOOP    LDAA 0,X    ; Get a source character
               STAA 0,Y    ; Copy it to the destination
               BEQ STRCPY_EXIT ; If it was the null, then exit
               INX         ; Else increment the pointers
               INY
               BRA STRCPY_LOOP ; and do it again
```

```

STRCPY_EXIT    PULA        ; Restore the registers
                PULY
                PULX
                RTS

```

```

;-----
;                               Guider LEDs ON                               |
; This routine enables the guider LEDs so that readings of the sensor      |
; correspond to the ?illuminated? situation.                                |
; Passed: Nothing                                                            |
; Returns: Nothing                                                            |
; Side: PORTA bit 5 is changed                                                |
G_LEDS_ON      BSET PORTA,%00100000 ; Set bit 5
               RTS                               ;|

```

```

;-----
;                               Guider LEDs OFF                               |
; This routine disables the guider LEDs. Readings of the sensor                |
; correspond to the ?ambient lighting? situation.                             |
; Passed: Nothing                                                                |
; Returns: Nothing                                                                |
; Side: PORTA bit 5 is changed                                                    |
G_LEDS_OFF      BCLR PORTA,%00100000 ; Clear bit 5
                RTS                               ;

```



```

DP_FRONT_SENSOR EQU TOP_LINE+3
DP_PORT_SENSOR  EQU BOT_LINE+0
DP_MID_SENSOR   EQU BOT_LINE+3
DP_STBD_SENSOR  EQU BOT_LINE+6
DP_LINE_SENSOR  EQU BOT_LINE+9

```

```

DISPLAY_SENSORS LDAA SENSOR_BOW      ; Get the FRONT sensor value
                JSR  BIN2ASC          ; Convert to ascii string in D
                LDX  #DP_FRONT_SENSOR ; Point to the LCD buffer position
                STD  0,X              ; and write the 2 ascii digits there
                LDAA SENSOR_PORT      ; Repeat for the PORT value
                JSR  BIN2ASC
                LDX  #DP_PORT_SENSOR
                STD  0,X
                LDAA SENSOR_MID       ; Repeat for the MID value
                JSR  BIN2ASC
                LDX  #DP_MID_SENSOR
                STD  0,X
                LDAA SENSOR_STBD      ; Repeat for the STARBOARD value
                JSR  BIN2ASC
                LDX  #DP_STBD_SENSOR
                STD  0,X
                LDAA SENSOR_LINE      ; Repeat for the LINE value
                JSR  BIN2ASC
                LDX  #DP_LINE_SENSOR
                STD  0,X
                LDAA #CLEAR_HOME      ; Clear the display and home the cursor
                JSR  cmd2LCD           ; "
                LDY  #40              ; Wait 2 ms until "clear display" command is complete
                JSR  del_50us
                LDX  #TOP_LINE        ; Now copy the buffer top line to the LCD
                JSR  putsLCD
                LDAA #LCD_SEC_LINE    ; Position the LCD cursor on the second line
                JSR  LCD_POS_CRSR
                LDX  #BOT_LINE        ; Copy the buffer bottom line to the LCD
                JSR  putsLCD
                RTS

```

```

;*****
;
;*          Update Display (Battery Voltage + Current State)          *
;*****
UPDT_DISPL     MOVB  #$90,ATDCTL5    ; R-just., uns., sing. conv., mult., ch=0, start
                BRCLR ATDSTAT0,$80,* ; Wait until the conver. seq. is complete
                LDAA  ATDDR0L        ; Load the ch0 result - battery volt - into A

```

```

LDAB  #39          ;AccB = 39
MUL           ;AccD = 1st result x 39
ADDD  #600         ;AccD = 1st result x 39 + 600
JSR   int2BCD
JSR   BCD2ASC
LDAA  #$8D         ;move LCD cursor to the 1st row, end of msg1
JSR   cmd2LCD
LDAA  TEN_THOUS     ;output the TEN_THOUS ASCII character
JSR   putcLCD
LDAA  THOUSANDS     ;output the THOUSANDS character
JSR   putcLCD
LDAA  #'.'         ; add the decimal place
JSR   putcLCD      ; put the dot into LCD
LDAA  HUNDREDS      ;output the HUNDREDS ASCII character
JSR   putcLCD      ;same for THOUSANDS, ?.? and HUNDREDS
LDAA  #$C7         ; Move LCD cursor to the 2nd row, end of msg2
JSR   cmd2LCD      ;
LDAB  CRNT_STATE   ; Display current state
LSLB           ; "
LSLB           ; "
LSLB
LDX   #tab         ; "
ABX           ; "
JSR   putsLCD      ; "
RTS

```

```

*****
,

```

```

ENABLE_TOF    LDAA  #%10000000
               STAA  TSCR1      ; Enable TCNT
               STAA  TFLG2      ; Clear TOF
               LDAA  #%10000100 ; Enable TOI and select prescale factor equal to 16
               STAA  TSCR2
               RTS

```

```

TOF_ISR      INC   TOF_COUNTER
               LDAA  #%10000000 ; Clear
               STAA  TFLG2      ; TOF
               RTI

```

```

*****
,

```

```

.*           Utility Subroutines          *
,
*****
,

```

```

initLCD:      BSET  DDRB,%11111111 ; configure pins PS7,PS6,PS5,PS4 for output
               BSET  DDRJ,%11000000 ; configure pins PE7,PE4 for output

```

```

LDY  #2000
JSR  del_50us
LDAA #$28
JSR  cmd2LCD
LDAA #$0C
JSR  cmd2LCD
LDAA #$06
JSR  cmd2LCD
RTS

```

```

;*****
;

```

```

clrLCD:    LDAA #$01
           JSR  cmd2LCD
           LDY  #40
           JSR  del_50us
           RTS

```

```

;*****
;

```

```

del_50us    PSHX           ; (2 E-clk) Protect the X register
eloop      LDX  #300       ; (2 E-clk) Initialize the inner loop counter
iloop      NOP            ; (1 E-clk) No operation
           DBNE X,iloop    ; (3 E-clk) If the inner cntr not 0, loop again
           DBNE Y,eloop    ; (3 E-clk) If the outer cntr not 0, loop again
           PULX           ; (3 E-clk) Restore the X register
           RTS            ; (5 E-clk) Else return

```

```

;*****
;

```

```

cmd2LCD:    BCLR LCD_CNTR, LCD_RS ; select the LCD instruction
           JSR  dataMov      ; send data to IR
           RTS

```

```

;*****
;

```

```

putsLCD:    LDAA 1,X+      ; get one character from string
           BEQ  donePS     ; get NULL character
           JSR  putcLCD
           BRA  putsLCD

```

```

donePS      RTS

```

```

;*****
;

```

```

putcLCD:    BSET LCD_CNTR, LCD_RS ; select the LCD data register (DR)c
           JSR  dataMov      ; send data to DR
           RTS

```

```

*****
,
dataMov:      BSET LCD_CNTR, LCD_E ; pull LCD E-signal high
              STAA LCD_DAT          ; send the upper 4 bits of data to LCD
              BCLR LCD_CNTR, LCD_E ; pull the LCD E-signal low to complete write oper.
              LSLA                  ; match the lower 4 bits with LCD data pins
              LSLA                  ; ""
              LSLA                  ; ""
              LSLA                  ; ""
              BSET LCD_CNTR, LCD_E ; pull LCD E-signal high
              STAA LCD_DAT          ; send the lower 4 bits of data to LCD
              BCLR LCD_CNTR, LCD_E ; pull the LCD E-signal low to complete write oper.
              LDY #1                ; adding this delay allows
              JSR del_50us          ; completion of most instructions
              RTS

```

```

*****
,
initAD      MOVB #$C0,ATDCTL2 ;power up AD, select fast flag clear
            JSR del_50us      ;wait for 50 us
            MOVB #$00,ATDCTL3 ;8 conversions in a sequence
            MOVB #$85,ATDCTL4 ;res=8, conv-clks=2, prescal=12
            BSET ATDDIEN,$0C   ;configure pins AN03,AN02 as digital inputs
            RTS

```

```

*****
,
.*          INTEGER To ASCII Conversion Routine      *
,
*****
,

```

```

int2BCD      XGDX                ;Save the binary number into .X
            LDAA #0              ;Clear the BCD_BUFFER
            STAA TEN_THOUS
            STAA THOUSANDS
            STAA HUNDREDS
            STAA TENS
            STAA UNITS
            STAA BCD_SPARE
            STAA BCD_SPARE+1
            CPX #0              ; Check for a zero input
            BEQ CON_EXIT        ; and if so, exit
            XGDX                ; Not zero, get the binary number back to .D as dividend
            LDX #10             ; Setup 10 (Decimal!) as the divisor
            IDIV                ; Divide Quotient is now in .X, remainder in .D
            STAB UNITS          ; Store remainder
            CPX #0              ; If quotient is zero,

```

```

    BEQ CON_EXIT      ; then exit
    XGDX              ; else swap first quotient back into .D
    LDX #10           ; and setup for another divide by 10
    IDIV
    STAB TENS
    CPX #0
    BEQ CON_EXIT
    XGDX              ; Swap quotient back into .D
    LDX #10           ; and setup for another divide by 10
    IDIV
    STAB HUNDREDS
    CPX #0
    BEQ CON_EXIT
    XGDX              ; Swap quotient back into .D
    LDX #10           ; and setup for another divide by 10
    IDIV
    STAB THOUSANDS
    CPX #0
    BEQ CON_EXIT
    XGDX              ; Swap quotient back into .D
    LDX #10           ; and setup for another divide by 10
    IDIV
    STAB TEN_THOUS

```

```

CON_EXIT    RTS      ; Were done the conversion

```

```

LCD_POS_CRCSR  ORAA #%10000000    ; Set the high bit of the control word
               JSR cmd2LCD          ; and set the cursor address
               RTS

```

```

*****
;
;*      BINARY To ASCII Conversion Routine      *
;
*****

```

```

BIN2ASC      PSHA      ; Save a copy of the input number
               TAB
               ANDB #%00001111    ; Strip off the upper nibble
               CLRA              ; D now contains 000n where n is the LSnibble
               ADDD #HEX_TABLE    ; Set up for indexed load
               XGDX
               LDAA 0,X           ; Get the LSnibble character
               PULB              ; Retrieve the input number into ACCB
               PSHA              ; and push the LSnibble character in its place

```

```

RORB          ; Move the upper nibble of the input number
RORB          ; into the lower nibble position.
RORB
RORB
ANDB #00001111 ; Strip off the upper nibble
CLRA          ; D now contains 000n where n is the MSnibble
ADDD #HEX_TABLE ; Set up for indexed load
XGDX
LDAA 0,X      ; Get the MSnibble character into ACCA
PULB         ; Retrieve the LSnibble character into ACCB
RTS

```

```

*****
;
;*          BCD to ASCII Conversion Routine          *
;
*****

```

```

BCD2ASC      LDAA #0          ; Initialize the blanking flag
             STAA NO_BLANK

```

```

C_TTHOU      LDAA TEN_THOUS   ; Check... (6 KB left)
             ORAA NO_BLANK
             BNE NOT_BLANK1

```

```

ISBLANK1     LDAA #' '       ; It's blank
             STAA TEN_THOUS   ; so store a space
             BRA C_THOU      ; and check the ?thousands? digit

```

```

NOT_BLANK1   LDAA TEN_THOUS   ; Get the ?ten_thousands? digit
             ORAA #$30        ; Convert to ascii
             STAA TEN_THOUS
             LDAA #$1         ; Signal that we have seen a ?non-blank? digit
             STAA NO_BLANK

```

```

C_THOU       LDAA THOUSANDS   ; Check the thousands digit for blankness
             ORAA NO_BLANK    ; If it's blank and ?no-blank? is still zero
             BNE NOT_BLANK2

```

```

ISBLANK2     LDAA #' '       ; Thousands digit is blank
             STAA THOUSANDS   ; so store a space
             BRA C_HUNS      ; and check the hundreds digit

```

```

NOT_BLANK2   LDAA THOUSANDS   ; (similar to ?ten_thousands? case)
             ORAA #$30

```

```

    STAA THOUSANDS
    LDAA #$1
    STAA NO_BLANK

```

```

C_HUNS      LDAA  HUNDREDS    ; Check the hundreds digit for blankness
             ORAA  NO_BLANK    ; If it's blank and ?no-blank? is still zero
             BNE   NOT_BLANK3

```

```

ISBLANK3     LDAA  #' '       ; Hundreds digit is blank
             STAA  HUNDREDS    ; so store a space
             BRA   C_TENS      ; and check the tens digit

```

```

NOT_BLANK3   LDAA  HUNDREDS    ; (similar to ?ten_thousands? case)
             ORAA  #$30
             STAA  HUNDREDS
             LDAA  #$1
             STAA  NO_BLANK

```

```

C_TENS       LDAA  TENS        ; Check the tens digit for blankness
             ORAA  NO_BLANK    ; If it's blank and ?no-blank? is still zero
             BNE   NOT_BLANK4

```

```

ISBLANK4     LDAA  #' '       ; Tens digit is blank
             STAA  TENS        ; so store a space
             BRA   C_UNITS     ; and check the units digit

```

```

NOT_BLANK4   LDAA  TENS        ; (similar to ?ten_thousands? case)
             ORAA  #$30
             STAA  TENS

```

```

C_UNITS      LDAA  UNITS       ; No blank check necessary, convert to ascii.
             ORAA  #$30
             STAA  UNITS
             RTS               ; We're done

```

```

*****
,
,*      Display the battery voltage      *
,
*****
,

```

```

    LDAA  #$C7                ; Move LCD cursor to the 2nd row, end of msg2
    JSR   cmd2LCD              ;
    LDAB  CRNT_STATE          ; Display current state
    LSLB                      ; "

```



```

LSLB          ; "
LSLB
LDX  #tab     ; "
ABX          ; "
JSR  putsLCD  ; "
RTS

```

```

*****
;
.*          Interrupt Vectors          *
;
*****
;
ORG  $FFFE
DC.W  Entry ; Reset Vector
ORG  $FFDE
DC.W  TOF_ISR ; Timer Overflow Interrupt Vector

```